# ForceDelta-VLA: Supplementary Material

Anonymous Authors

## I. Training and Architecture Details

The teacher is trained for 50,000 steps with AdamW at a peak learning rate of $2 \times 10^{-5}$. It predicts $H$=50 actions from the latest 100 ms of the 100 Hz wrench stream, and flow-matching inference uses $N_{\text{flow}}$=10 steps.

After freezing the teacher, we train the rank-32 force-agnostic adapter for 10,000 steps at a learning rate of $10^{-4}$. The visual–language prefix is pooled into $M$=16 positional bins and projected to $Q$=2 task-context tokens. The correction policy contains a causal force encoder and one attention block that processes the condition tokens. Layer normalization precedes attention, and separate force and delay output projections are initialized to zero. It predicts $K$=5 corrections and is trained for 30,000 steps at a learning rate of $3 \times 10^{-4}$ under the sampled asynchronous schedule. The delay-correction pass masks the force-token position in the attention-validity mask.

### A. Action Representation and Context Encoding

Pose actions use $\mathcal{P}(A) = [p; r]$, where $p \in \mathbb{R}^3$ is translation and $r \in \mathbb{R}^6$ is the continuous rotation representation. Pose actions and wrench measurements use the corresponding robot-base frame. Differences are scaled by the training-set action standard deviations. Rotational corrections are added in the teacher's 6D action coordinates and mapped to valid rotation matrices by Gram–Schmidt orthogonalization. Bimanual inputs and outputs concatenate the per-arm quantities; correction construction is component-wise and bounds are applied independently to each arm.

The visual–language prefix $E_k$ excludes state, force, and action tokens. Its mask $m_k$ selects available image-view and non-padding language tokens. Masked mean pooling partitions the padded prefix into $M = 16$ consecutive groups of approximately equal size:

$$(\bar{E}_k, \bar{m}_k) = \text{Pool}_M(E_k, m_k), \qquad Z_k^{\text{ctx}} = P_{\text{ctx}}(\bar{E}_k, \bar{m}_k). \tag{1}$$

The learned projector uses $Q = 2$ queries to aggregate valid pooled tokens. Condition-specific projections map task context, force history, state, reference-action segment, and timing features to the shared token width. Learned type embeddings identify the conditions; no positional encoding is used in the conditioning set. Reordering condition tokens together with their type embeddings does not change the output at the fixed correction-query token. The force and delay passes share the attention block and use separate output projections; the delay pass masks the force token.

Cache age and interpolation phase are represented as

$$\xi_{t,k} = \left[\min\left(\frac{t - t_k}{c}, 1\right); \alpha_{t,k}\right], \qquad c = 0.6 \text{ s}, \tag{2}$$

where $\alpha_{t,k}$ is the position of $t$ between adjacent reference-action samples.

The correction policy observes the complete $K$-step reference-action segment aligned with its output timestamps. The segment is flattened and projected to one reference-action token with a type embedding, so each predicted correction is conditioned on the corresponding future reference-action waypoint without increasing the attention-sequence length.

At deployment, force and delay corrections are clipped separately and element-wise before composition. The implementation accepts either nine-dimensional nonnegative bounds $b_F, b_S \in \mathbb{R}^9_{\geq 0}$ or scalar values broadcast over the pose coordinates; bimanual clipping is applied independently per arm. The final robot configuration must specify the values used for $b_F$ and $b_S$.

### B. Reference-State Alignment

The action representation and rotational correction composition are defined in Sec. III-A of the main paper. Let $\widetilde{S}^p$ be the nine per-arm pose coordinates of the normalized robot state, obtained after converting orientation to the continuous 6D representation, and let $\sigma_S^p$ and $\sigma_A^p$ be the corresponding training-set state and action standard deviations. Reference-state alignment expresses the current prediction relative to the reference-query state $S_k$. The implementation computes

$$\Gamma(S_t, S_k) = \frac{\sigma_S^p + \varepsilon}{\sigma_A^p + \varepsilon} \odot (\widetilde{S}_t^p - \widetilde{S}_k^p), \qquad \varepsilon = 10^{-6}. \tag{3}$$

This difference, rescaled separately for each coordinate, is included in the learned delay-correction target, placing the current force-agnostic prediction and reference action in the same normalized delta-action coordinate system. The composed normalized action is de-normalized, applied relative to the reference-action-query state $S_k$, and projected to a valid rotation by Gram–Schmidt orthogonalization.

Let $\mathcal{R}(S, a)$ convert a relative action $a$ based at state $S$ into an absolute robot command. The paired targets account for the change in reference state at each action step:

$$\begin{aligned} &\mathcal{R}\Big(S_k, \mathcal{P}(A_k^{\text{ref}}(t_j)) + \Delta A_{T,t|k,j}^{\text{force}} + \Delta A_{T,k\to t,j}^{\text{delay}}\Big) \\ &\quad \approx \mathcal{R}\Big(S_t, \mathcal{P}(A_{T,t|k,j}^{\text{cond}})\Big). \end{aligned} \tag{4}$$

Before clipping, substitution recovers the force-conditioned chunk expressed relative to $S_k$. The alignment relation holds in the shared translation-plus-6D representation up to the $\varepsilon$ stabilization term; Gram–Schmidt projection is applied after restoring the action delta.

### C. Asynchronous Schedule Replay

Correction-policy training examples use the 30 Hz state/action timestamps while retaining every 100 Hz wrench sample in $\mathcal{F}_t$. The sampled schedule independently draws inter-query periods $T_i \sim \mathcal{U}(1/15, 1/5)$ s and reference-action-query latencies $L_i \sim \mathcal{U}(0.05, 0.30)$ s. Query samples are first selected greedily from the recorded timestamps. We then add random offsets $J_i \sim \mathcal{U}(-0.01, 0.01)$ s so that reference actions are interpolated between recorded samples. Correction queries may start more frequently at deployment; the actual correction-update rate is measured directly rather than inferred from the action-sample rate within a chunk.

The schedule is sampled before target extraction and kept fixed during correction-policy optimization. Let $t_i$ and $\bar{t}_i = t_i + L_i$ denote the start and availability times of reference query $i$. At a sampled correction time $t$, the active reference index is

$$k(t) = \arg\max_{i:\bar{t}_i \le t} t_i. \tag{5}$$

This reference supplies the cached pooled context, interpolated reference-action segment, and timestamps for the timing features and paired targets. The schedule determines the reference age, interpolation phase, and query latency encountered during training.

### D. Force-Agnostic Mode Optimization

The learned force-agnostic token and mode-specific adapter are active only for force-agnostic queries. With the force-conditioned teacher parameters $\theta$ fixed, the reference-action parameters $\phi$ are optimized using the teacher's original flow-matching objective,

$$\mathcal{L}_{\text{ref}} = \mathbb{E}_{\mathcal{D},\eta,\epsilon}\left[\ell_{\text{FM}}\left(T^{\text{ref}}_{\theta,\phi}, A^{*}_{t:t+H-1}; \eta, \epsilon\right)\right], \tag{6}$$

where $\eta$ is flow time and $\epsilon$ is the initial flow noise. Force-conditioned and force-agnostic predictions used to construct a correction target share the same $\epsilon$.

### E. Correction-Loss Weighting

For a $K$-step correction chunk, the normalized temporal weights are

$$w_j = \frac{\beta^{j-1}}{\sum_{m=1}^{K} \beta^{m-1}}, \qquad 0 < \beta \le 1. \tag{7}$$

Thus, $\beta < 1$ emphasizes near-term corrections, $\beta = 1$ weights all steps equally, and $\sum_j w_j = 1$ keeps the loss scale stable as $K$ changes. The loss is a mean-squared error in normalized pose-action coordinates.

## II. Asynchronous Reference-Action and Correction Execution

### A. Reference-Action Updates

A reference-action query $k$, started at $t_k$ from state $S_k$, stores the reference-action update $(A^{\text{ref}}_{T,k,1:H}, \bar{E}_k, \bar{m}_k, S_k, t_k, \bar{t}_k)$ when its result becomes available at $\bar{t}_k$. This stored update is kept unchanged. Its samples have intended execution times

$$t^{\text{ref}}_{k,n} = t_k + (n-1)/f_A, \qquad n = 1, \dots, H. \tag{8}$$

At time $t$, the active reference action is the completed query with the latest start time,

$$k^*(t) = \arg\max_{k:\,\bar{t}_k \le t} t_k. \tag{9}$$

An older query that completes later therefore cannot replace a newer active reference-action update. Samples whose intended times precede $\bar{t}_{k^*}$ are skipped for direct execution; the cached chunk itself is not truncated. For $t \ge \bar{t}_{k^*}$, the reference pose $A^{\text{ref}}_{k^*}(t)$ is obtained by timestamp-based interpolation over the original timestamped chunk.

### B. Correction Updates

A correction query $q$, started at $t^F_q$, reads $k_q = k^*(t^F_q)$ together with $(\mathcal{F}_{t^F_q}, S_{t^F_q})$. It completes at $\bar{t}^F_q$ with force-conditioned and delay correction chunks. Their samples are timestamped as

$$t^F_{q,j} = t^F_q + (j-1)/f_A, \qquad j = 1, \dots, K. \tag{10}$$

Here $f_A$ defines the spacing between correction samples within a chunk; correction query start times $t^F_q$ are scheduled independently. Step $j$ is held over $[t^F_{q,j}, t^F_{q,j} + 1/f_A)$, and correction samples are not interpolated. At control time $t$, the executor selects the most recently started correction query whose result has completed and remains valid, together with its active step:

$$\begin{aligned} q^*(t) &= \arg\max_{q:\,\bar{t}^F_q \le t < t^F_q + K/f_A} t^F_q, \\ j(t; q^*) &= 1 + \left\lfloor (t - t^F_{q^*}) f_A \right\rfloor. \end{aligned} \tag{11}$$

Any initial correction steps whose execution times have passed during inference are therefore skipped, while the remaining valid steps execute sequentially until a newer correction chunk completes. Let $j^* = j(t; q^*)$. Within the selected interval, Eq. (1) of the main paper is evaluated using the selected correction query time and active step:

$$\begin{aligned} \mathcal{P}(A^{\text{cmd}}(t)) &\equiv \mathcal{P}(A^{\text{cmd}}_{t^F_{q^*}, j^*}) \\ &= \mathcal{P}\left(A^{\text{ref}}_{k_{q^*}}(t^F_{q^*,j^*})\right) + \Delta\hat{A}^{\text{force}}_{q^*,j^*} + \Delta\hat{A}^{\text{delay}}_{q^*,j^*}. \end{aligned} \tag{12}$$

The two corrections are clipped independently and element-wise in normalized delta-action coordinates before composition. The composed command remains referenced to $S_{k_{q^*}}$ and is converted to an absolute robot command as described above. A correction chunk that completes after $t^F_q + K/f_A$ is rejected. If no valid correction chunk remains, the executor uses the platform's safe hold or abort policy. The final implementation record will specify gripper sampling and behavior beyond the last valid reference-action timestamp.

## III. Additional Evaluation Protocols

### A. Task Success Criteria

The task-specific success criteria are summarized in Table I.

TABLE I
TASK SUCCESS CRITERIA. RED ENTRIES ARE THRESHOLDS THAT STILL NEED TO BE CONFIRMED.

| **Platform** | **Task** | **Success criterion** |
|---|---|---|
| Single-arm | Object Flipping | Object rotated by at least $150^\circ$ and left stable |
| Single-arm | USB Insertion | Connector inserted to within 2 mm of its final depth |
| Single-arm | Cabinet Opening | Door opened beyond $60^\circ$ |
| Single-arm | Button Pressing | Button fully pressed |
| Single-arm | Whiteboard Wiping | At least $90\%$ of the marked region removed |
| Bimanual | Plug Removal | Plug fully separated without dropping either component |
| Bimanual | Plug Insertion | Plug fully seated while the second arm stabilizes the socket |
| Bimanual | Drawer Opening | Drawer opened by at least 15 cm |
| Bimanual | Whiteboard Wiping | At least $90\%$ of the marked region removed while the left arm stabilizes the board and the right arm wipes |

### *B. Metric Aggregation*

As in the main paper, peak force and completion time are reported as mean $\pm$ standard deviation over successful trials pooled across tasks within each platform. Consequently, a task's contribution to the pooled metrics depends on the method's number of successes on that task. Success rate is averaged across the nine tasks, with 20 trials per method and task. The precise per-trial force reduction across arms and completion-time boundaries must be verified against the final evaluation code.

### *C. Task-Wise Trial Counts*

The final experiment record will report the successful-trial count for each method and task. Each count is out of twenty evaluation trials and aggregates to the main paper's success-rate table.